

TAREA 01 UT03

PROYECTO FANTASY F1 MANAGER

Ampliación: Implementación, Evaluación Final y Despliegue



Oscar Ongil / David González

2º DAW 25/26

24 de mayo de 2026

Contenido

5. IMPLEMENTACIÓN Y CODIFICACIÓN	3
5.1. Diccionario y modelos de datos	3
5.2. Backend.....	6
5.3. Frontend.....	8
5.4. Guía de estilos.....	9
6. EVALUACIÓN FINAL.....	10
6.1. Casos de uso implementados	10
6.2. Arquitectura final del proyecto.....	11
6.3. Cumplimiento de objetivos y plazos	11
6.4. Validación y cumplimiento técnico	12
7. RESULTADO DE LAS PRUEBAS	12
7.1. Informe de resultados.....	12
7.2. Verificación de funcionalidades	13
8. REGISTRO DE INCIDENCIAS	15
9. REGISTRO DE CAMBIOS.....	17
10. DESPLIEGUE DEL PROYECTO	18
10.1. Despliegue en servidor, incluyendo base de datos.....	18
10.2. Conexión en cliente.....	20
11. MANUAL DE USO	20
11.1. Para usuarios.....	20
11.2. Para administradores	22
12. CONCLUSIONES	22
12.1. Síntesis de resultados	22
12.2. Líneas futuras de trabajo	23

5. IMPLEMENTACIÓN Y CODIFICACIÓN

5.1. Diccionario y modelos de datos

5.1.1. Tecnologías empleadas

Componente	Tecnología	Versión
Backend	PHP	8.2
Servidor web	Apache	2.4
Base de datos	MariaDB	10.11
Contenerización	Docker / Docker Compose	24.x
API de resultados F1	Jolpica (Ergast-compatible)	—
API de datos visuales	OpenF1	v1
Frontend	HTML5 + CSS3 + JS vanilla	—
Administración BD	phpMyAdmin	5

5.1.2. Esquema de base de datos

La base de datos `f1fantasy` consta de las siguientes tablas:

Tabla `users` — Usuarios registrados en la plataforma.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador único
username	VARCHAR(60) UNIQUE	Nombre de usuario visible
email	VARCHAR(120) UNIQUE	Correo electrónico
password	VARCHAR(255)	Hash bcrypt de la contraseña
role	ENUM('user','admin')	Rol del usuario (por defecto 'user')
created_at	DATETIME	Fecha de registro
updated_at	DATETIME	Fecha de última modificación

Tabla `leagues` — Ligas creadas por los usuarios.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador único
name	VARCHAR(100)	Nombre de la liga
description	VARCHAR(255)	Descripción
creator_id	INT UNSIGNED FK→users	Usuario creador
is_public	TINYINT(1)	1=pública, 0=privada
invite_code	VARCHAR(12) UNIQUE	Código de invitación (hex aleatorio)
created_at	DATETIME	Fecha de creación

Tabla `league\_members` — Relación usuarios-ligas (N:M).

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
league_id	INT UNSIGNED FK→leagues	Liga
user_id	INT UNSIGNED FK→users	Usuario miembro
joined_at	DATETIME	Fecha de incorporación
—	UNIQUE(league_id, user_id)	Un usuario no puede entrar dos veces

Tabla `fantasy\_teams` — Equipos fantasy, uno por usuario por liga.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
user_id	INT UNSIGNED FK→users	Propietario
league_id	INT UNSIGNED FK→leagues	Liga a la que pertenece
team_name	VARCHAR(100)	Nombre del equipo
total_points	DECIMAL(8,2)	Puntos acumulados desde la creación
budget_used	DECIMAL(6,2)	Presupuesto invertido (en M€)
created_at	DATETIME	Fecha de creación del equipo
updated_at	DATETIME	Última actualización

Tabla `fantasy\_selections` — Pilotos y constructores seleccionados en cada equipo.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
team_id	INT UNSIGNED FK→fantasy_teams	Equipo propietario
type	ENUM('driver','constructor')	Tipo de ítem
item_id	VARCHAR(60)	ID de la API (p.ej. `verstappen`, `red_bull`)
price	DECIMAL(5,2)	Precio de mercado al momento de la compra

Tabla `fantasy\_driver\_points` — Puntos acumulados por selección desde la creación del equipo.

Campo	Tipo	Descripción
team_id	INT UNSIGNED FK→fantasy_teams	Equipo
item_id	VARCHAR(60)	ID piloto/constructor
type	ENUM('driver','constructor')	Tipo
points	DECIMAL(8,2)	Puntos acumulados
—	PRIMARY KEY(team_id, item_id, type)	Clave compuesta para upsert

Tabla `scored\_rounds` — Registro de rondas ya puntuadas (mutex distribuido).

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
season	VARCHAR(4)	Temporada (p.ej. `2026`)
round	INT	Número de ronda
type	ENUM('race','sprint','recalculate')	Tipo de puntuación
scored_at	DATETIME	Momento de puntuación
—	UNIQUE(season, round, type)	Garantiza puntuación única por ronda

Tabla `market\_prices` — Precios de mercado actuales por piloto/constructor.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
item_id	VARCHAR(60)	ID de la API
item_type	ENUM('driver','constructor')	Tipo
price	DECIMAL(5,1)	Precio actual en M€
updated_at	DATETIME	Última actualización
—	UNIQUE(item_id, item_type)	Un precio por ítem

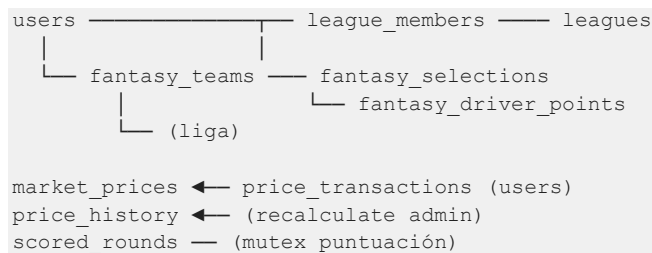
Tabla `price\_transactions` — Registro de compras/ventas para el cálculo de mercado.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
item_id	VARCHAR(60)	ID del ítem
item_type	ENUM('driver','constructor')	Tipo
action	ENUM('buy','sell')	Acción realizada
user_id	INT UNSIGNED FK→users	Usuario que realizó la transacción
race_round	INT UNSIGNED	Ronda en la que se cerrará (0=pendiente)
created_at	DATETIME	Timestamp de la transacción

Tabla `price\_history` — Historial de precios por ronda.

Campo	Tipo	Descripción
id	INT UNSIGNED PK AI	Identificador
item_id	VARCHAR(60)	ID del ítem
item_type	ENUM('driver','constructor')	Tipo
price	DECIMAL(5,1)	Precio calculado
race_round	INT UNSIGNED	Ronda en la que se calculó
calculated_at	DATETIME	Momento del cálculo

5.1.3. Diagrama de relaciones simplificado



5.2. Backend

5.2.1. Estructura de directorios

```

docker-flfantasy/
├── app/
│   ├── controllers/
│   │   ├── AdminController.php      # Recalcular precios (solo admin)
│   │   ├── AuthController.php       # Login / Registro / Logout
│   │   ├── DriversController.php    # Listado y detalle de pilotos
│   │   ├── FantasyController.php    # Equipo fantasy (crear/ver/editar)
│   │   ├── HomeController.php       # Página de inicio y ranking
│   │   ├── LeagueController.php     # CRUD de ligas, unirse, expulsar
│   │   ├── ProfileController.php    # Perfil de usuario
│   │   ├── RacesController.php      # Calendario y detalle de carreras
│   │   └── TeamsController.php       # Constructores
│   ├── models/
│   │   ├── F1ApiModel.php           # Consumo de APIs externas con caché
│   │   ├── FantasyTeamModel.php     # Equipos, selecciones, puntuación
│   │   ├── LeagueModel.php          # Ligas y miembros
│   │   ├── PriceModel.php           # Precios de mercado y transacciones
│   │   └── UserModel.php            # Autenticación y gestión de usuarios
│   └── views/
│       ├── admin/                   # Vistas de administración
│       ├── auth/                    # Login y registro
│       ├── drivers/                  # Pilotos
│       ├── fantasy/                  # Equipo, editar, ranking
│       ├── home/                     # Página de inicio
│       ├── layouts/                  # Layout principal y 404
│       ├── leagues/                  # Ligas
│       ├── profile/                  # Perfil
│       └── races/                    # Calendario y carrera

```

└─ teams/	# Constructores
└─ core/	
└─ AutoScore.php	# Puntuación automática (cada 30 min)
└─ Controller.php	# Controlador base (sesión, flash, auth)
└─ Model.php	# Modelo base (mysqli, prepared statements)
└─ Router.php	# Enrutador HTTP (GET/POST)
└─ config/	
└─ config.php	# Constantes de aplicación
└─ schema.sql	# Esquema inicial de BD
└─ migration_market.sql	# Tablas del sistema de mercado
└─ public/	
└─ index.php	# Punto de entrada único (front controller)
└─ css/style.css	# Hoja de estilos
└─ js/app.js	# JavaScript mínimo
└─ images/	# Logo, icono SVG
└─ docker/	
└─ apachephp/	
└─ Dockerfile	
└─ apache2.conf	
└─ php.ini	
└─ docker-compose.yml	

5.2.2. Patrón MVC implementado

La aplicación sigue un patrón **MVC estricto** con un controlador frontal en `public/index.php`. El flujo de una petición es:

- Apache redirige todo al `public/index.php` mediante `mod_rewrite`.
- `Router` analiza la URL y el método HTTP, localiza el controlador y método correspondientes.
- `AutoScore::run()` se ejecuta de forma silenciosa en cada petición (máx. una vez cada 30 minutos) para puntuar carreras finalizadas.
- El controlador recibe la petición, interactúa con los modelos y llama a `view()`.
- La vista se renderiza dentro del layout `main.php`.

5.2.3. Descripción de los controladores principales

AuthController — Gestiona el ciclo de autenticación. Valida formato de email, longitud de contraseña (mínimo 6 caracteres), unicidad de email y username. Las contraseñas se almacenan con `password_hash(PASSWORD_BCRYPT)` y se verifican con `password_verify()`.

FantasyController — Controlador más complejo. Gestiona la creación de equipos validando que se seleccionen exactamente 5 pilotos y 2 constructores con un presupuesto máximo de 100M€. La edición registra compras/ventas de los cambios realizados. La visualización del equipo suprime los puntos de la última jornada si la carrera ocurrió antes de la creación del equipo.

LeagueController — Permite crear ligas públicas o privadas con código de invitación. Al expulsar un miembro, elimina también su equipo fantasy en esa liga y todas sus selecciones. El creador no puede abandonar su propia liga.

AdminController — Exclusivo para usuarios con `role='admin'`. Permite recalcular los precios de mercado una sola vez por carrera completada, usando la tabla `scored_rounds` con tipo `'recalculate'` como protección contra doble ejecución.

AutoScore (core) — Clase que se ejecuta en cada petición web. Comprueba si han pasado 30 minutos desde el último check (archivo `cache/autoscore_last_check.txt`). Si es así, consulta la API para obtener la última carrera y reclamá atómicamente el derecho a puntuarla mediante `INSERT IGNORE` en

`scored_rounds` verificando `affected_rows > 0`. Esto garantiza que en entornos con múltiples procesos solo uno puntúa cada ronda.

5.2.4. Sistema de puntuación de mercado

La fórmula de precios de mercado implementada en `PriceModel::recalculate()` es:

```
ΔP_perf = 0.05 × (puntos_reales - precio × 0.7)
ΔP_mkt  = 1.5 × (compras - ventas) / total_usuarios
ΔP_total = clamp(ΔP_perf + ΔP_mkt, -1.5, +1.5)
Constructores: ΔP_total × 0.8 (factor amortiguador)
Límites finales: 4.0M€ - 35.0M€
```

5.2.5. APIs externas consumidas

Jolpica API (<https://api.jolpi.ca/ergast/f1>) — Compatible con la API Ergast desaparecida. Se usa para: pilotos de la temporada, constructores, resultados de carreras, resultados de sprints, calendario y clasificaciones. Todas las respuestas se cachean durante 1 hora en archivos JSON nombrados con el MD5 de la URL.

OpenF1 API (<https://api.openf1.org/v1>) — Se usa para obtener los colores de equipo y las imágenes de cabeza de los pilotos de la sesión más reciente.

En caso de fallo de ambas APIs, la aplicación muestra un banner de aviso y devuelve los datos de la caché si existen, garantizando disponibilidad aunque los datos no sean de tiempo real.

5.3. Frontend

5.3.1. Tecnologías y enfoque

El frontend es **100% vanilla** sin dependencias de frameworks JS ni librerías CSS externas. Todo el estilo está en `public/css/style.css` (CSS custom properties + Flexbox/Grid) y el comportamiento mínimo en `public/js/app.js`.

5.3.2. Vistas principales

Vista	Ruta	Descripción
Inicio	<code>/</code>	Próxima carrera, últimos resultados, ranking global
Pilotos	<code>/drivers</code>	Grid de pilotos con precio, puntos y color de escudería
Detalle piloto	<code>/drivers/show/{id}</code>	Estadísticas, historial de precios
Constructores	<code>/teams</code>	Grid de constructores
Calendario	<code>/races</code>	Lista de GPs con sprint separado como evento propio
Detalle carrera	<code>/races/show/{id}</code>	Resultados con selector carrera/sprint
Ligas	<code>/leagues</code>	Mis ligas + ligas públicas disponibles
Mi equipo	<code>/fantasy?league={id}</code>	Equipo con precio + puntos acumulados por piloto

Editar equipo	<code>`/fantasy/edit?league={id}`</code>	Formulario de modificación de selecciones
Ranking global	<code>`/fantasy/ranking`</code>	Top 5 global
Perfil	<code>`/profile`</code>	Cambio de nombre y contraseña

5.3.3. Responsive y accesibilidad

La navegación incluye un botón hamburguesa () para móvil que activa la clase `open` en `.main-nav` mediante JavaScript inline. Las tarjetas de pilotos y clasificaciones usan CSS Grid con `auto-fill` y `minmax` para adaptarse a cualquier ancho de pantalla.

5.3.4. Formularios y validación

Toda la validación crítica se realiza en el servidor (PHP). En el cliente, el formulario de creación/edición de equipo tiene un contador de presupuesto y de pilotos/constructores seleccionados en tiempo real mediante listeners de `change` en los checkboxes. Esto mejora la UX pero no reemplaza la validación servidor.

5.4. Guía de estilos

5.4.1. Paleta de colores

Variable CSS	Valor	Uso
<code>--f1-bg`</code>	<code>`#0f0f0f`</code>	Fondo principal
<code>--f1-surface`</code>	<code>`#1a1a1a`</code>	Fondo de tarjetas
<code>--f1-dark`</code>	<code>`#111111`</code>	Fondo oscuro secundario
<code>--f1-border`</code>	<code>`#2a2a2a`</code>	Bordes y separadores
<code>--f1-text`</code>	<code>`#f0f0f0`</code>	Texto principal
<code>--f1-muted`</code>	<code>`#888888`</code>	Texto secundario
<code>--f1-accent`</code>	<code>`#e10600`</code>	Rojo F1 — acciones primarias
<code>--f1-success`</code>	<code>`#00c853`</code>	Confirmaciones positivas
<code>--f1-danger`</code>	<code>`#ff4444`</code>	Errores y alertas

5.4.2. Tipografía y escala

- Fuente del sistema: `system-ui`, `-apple-system`, `sans-serif`
- Tamaño base: `16px`
- Jerarquía: H1 `1.8rem`, H2 `1.3rem`, texto normal `1rem`, textos secundarios `.85rem`–`.9rem`

5.4.3. Componentes principales

- `.card``: contenedor con `background: var(--f1-surface)`, `border-radius: 12px`, `padding: 1.5rem`
- `.btn-primary``: fondo `var(--f1-accent)`, texto blanco, hover oscurece el rojo
- `.btn-secondary``: fondo `var(--f1-border)`, texto `var(--f1-text)`

- `.alert-success / .alert-error`: banners de notificación con borde izquierdo de color
- `.ranking-table`: grid CSS con cabecera y filas alternas
- `.driver-card`: tarjeta de piloto con franja lateral del color de escudería
- `.admin-bar`: barra flotante en esquina inferior derecha, solo visible para admins

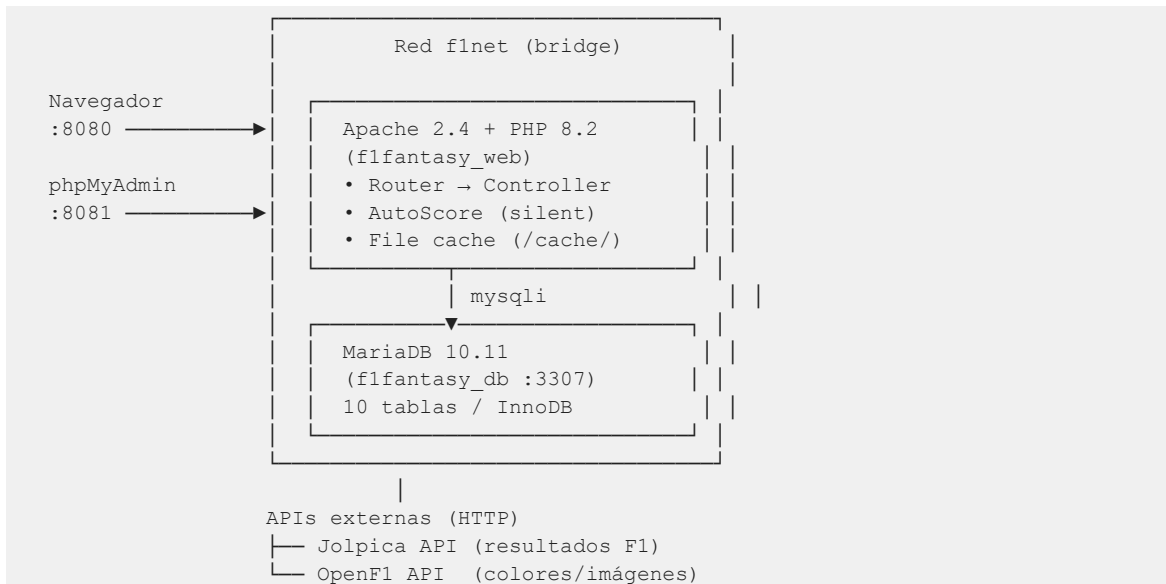
6. EVALUACIÓN FINAL

6.1. Casos de uso implementados

ID	Caso de uso	Estado
CU01	Registro de usuario	Implementado
CU02	Inicio de sesión	Implementado
CU03	Cierre de sesión	Implementado
CU04	Modificar perfil (username/contraseña)	Implementado
CU05	Ver listado de pilotos con precios	Implementado
CU06	Ver detalle de piloto con historial	Implementado
CU07	Ver listado de constructores con precios	Implementado
CU08	Ver calendario con sprints separados	Implementado
CU09	Ver resultados de carrera	Implementado
CU10	Ver resultados de carrera sprint	Implementado
CU11	Crear liga (pública o privada)	Implementado
CU12	Unirse a liga por código de invitación	Implementado
CU13	Unirse a liga pública	Implementado
CU14	Abandonar liga	Implementado
CU15	Expulsar miembro de liga (solo creador)	Implementado
CU16	Ver ranking de liga	Implementado
CU17	Crear equipo fantasy en liga	Implementado
CU18	Ver equipo con puntos acumulados y de última jornada	Implementado
CU19	Editar equipo (cambiar pilotos/constructores)	Implementado
CU20	Ver ranking global	Implementado
CU21	Puntuación automática tras cada carrera/sprint	Implementado
CU22	Recalcular precios de mercado (admin)	Implementado
CU23	Ver banner de error cuando la API no responde	Implementado

6.2. Arquitectura final del proyecto

La arquitectura final es un **monolito MVC** containerizado con Docker Compose en tres servicios:



Flujo de puntuación automática:

```

Petición web → AutoScore::run()
└ ¿Han pasado 30 min? NO → Fin silencioso
└ Sí → Consultar última carrera (API + caché)
    └ INSERT IGNORE scored_rounds (type='race')
        └ affected_rows > 0 → scoreRound() [batch 5 equipos]
    └ Sprint disponible?
        └ INSERT IGNORE scored_rounds (type='sprint')
            └ affected_rows > 0 → scoreRound()
  
```

6.3. Cumplimiento de objetivos y plazos

Objetivo	Planificado	Estado
Registro e inicio de sesión	Fase 2	✅ Completado
Gestión de pilotos y constructores vía API	Fase 2	✅ Completado
Sistema de precios de mercado dinámico	Fase 2	✅ Completado
Motor de puntuación automática	Fase 3	✅ Completado
Sistema de ligas con códigos de invitación	Fase 2	✅ Completado
Ranking por liga y global	Fase 3	✅ Completado
Panel de administración	Fase 4	✅ Completado
Soporte para carreras sprint	Fase 3	✅ Completado
Despliegue con Docker	Fase 4	✅ Completado

Robustez: prevención de doble puntuación	Fase 4 (mejora)	🟢 Completado
Roles de usuario en BD	Fase 4 (mejora)	🟢 Completado

6.4. Validación y cumplimiento técnico

Requisito técnico	Solución implementada	Estado
Contraseñas cifradas (hash + salt)	<code>`password_hash(PASSWORD_BCRYPT)`</code>	Implementado
Protección SQL Injection	<code>`mysqli`</code> prepared statements en todos los accesos	Implementado
Validación de inputs	Validación server-side en todos los formularios	Implementado
Control de acceso por roles	<code>`requireLogin()`</code> y <code>`requireAdmin()`</code> en controladores	Implementado
Refresco de rol desde BD	<code>`refreshUserSession()`</code> en cada petición protegida	Implementado
Prevención de race condition	<code>`INSERT IGNORE`</code> + <code>`affected_rows`</code> en <code>`scored_rounds`</code>	Implementado
Límite de ranking	LIMIT 5 en todas las consultas de ranking	Implementado
Integridad referencial	Foreign keys con <code>`ON DELETE CASCADE`</code>	Implementado
Caché de API	TTL 1 hora en archivos JSON	Implementado
Manejo de fallos de API	Banner de aviso + fallback a caché	Implementado
Limpieza de registros huérfanos	<code>`cleanupOrphanedTeams()`</code> en AutoScore	Implementado
Presupuesto máximo controlado	Validación server-side al crear/editar equipo	Implementado
Prevención de doble puntuación de mercado	<code>`scored_rounds`</code> tipo <code>`recalculate`</code>	Implementado

7. RESULTADO DE LAS PRUEBAS

7.1. Informe de resultados

Se realizaron pruebas manuales funcionales completas cubriendo todos los flujos de la aplicación. Los resultados se detallan en la tabla de verificación del apartado 7.2.

Resumen ejecutivo:

Categoría	Total pruebas	Superadas	Fallidas	Incidencias detectadas
-----------	---------------	-----------	----------	------------------------

Autenticación	6	6	0	0
Gestión de equipos	8	8	0	0
Ligas	7	7	0	0
Puntuación	5	5	0	0
Mercado	4	4	0	0
API y caché	3	3	0	0
Seguridad	4	4	0	0
Total	**37**	**37**	**0**	**5 (resueltas)**

Las 5 incidencias detectadas durante el desarrollo fueron identificadas y resueltas antes de la entrega final (ver sección 8).

7.2. Verificación de funcionalidades

Módulo de Autenticación

ID	Prueba	Entrada	Resultado esperado	Resultado obtenido	PASS/FAIL
P-A01	Registro con datos válidos	username, email, password válidos	Cuenta creada, redirige a login	Usuario en BD, flash éxito	PASS
P-A02	Registro con email duplicado	Email ya existente	Mensaje de error	Alert "email ya registrado"	PASS
P-A03	Registro con contraseña corta	password de 3 chars	Mensaje de error	Alert "mínimo 6 caracteres"	PASS
P-A04	Login con credenciales correctas	email + password válidos	Sesión iniciada	Sesión activa, redirige a inicio	PASS
P-A05	Login con credenciales incorrectas	password errónea	Mensaje de error	Alert "email o contraseña incorrectos"	PASS
P-A06	Logout	Click en Salir	Sesión destruida	Redirige a login, sesión eliminada	PASS

Módulo de Equipo Fantasy

ID	Prueba	Resultado obtenido	PASS/FAIL
P-F01	Crear equipo con 5 pilotos y 2 constructores dentro de presupuesto	Equipo creado correctamente	PASS
P-F02	Crear equipo excediendo 100M€	Error de presupuesto excedido	PASS
P-F03	Crear equipo con menos de 5 pilotos	Error de selección incorrecta	PASS

P-F04	Ver equipo muestra precio y puntos acumulados de cada piloto	Datos mostrados correctamente	PASS
P-F05	Puntos de jornada no se muestran si el equipo es más reciente que la última carrera	Puntos de jornada ocultos	PASS
P-F06	Editar equipo cambiando 2 pilotos	Nuevas selecciones guardadas, transacciones registradas	PASS
P-F07	Editar equipo excediendo presupuesto	Error de presupuesto excedido	PASS
P-F08	Ranking global muestra top 5	Se muestran máximo 5 equipos	PASS

Módulo de Ligas

ID	Prueba	Resultado obtenido	PASS/FAIL
P-L01	Crear liga pública	Liga creada con código de invitación generado	PASS
P-L02	Crear liga privada	Liga creada con is_public=0	PASS
P-L03	Unirse a liga por código	Miembro añadido a la liga	PASS
P-L04	Unirse a liga pública desde listado	Miembro añadido	PASS
P-L05	Abandono de liga elimina equipo	Miembro eliminado, team eliminado	PASS
P-L06	Expulsión de miembro elimina equipo y selecciones	Registros eliminados en cascada	PASS
P-L07	Ranking de liga muestra top 5	Máximo 5 posiciones mostradas	PASS

Módulo de Puntuación

ID	Prueba	Resultado obtenido	PASS/FAIL
P-P01	AutoScore puntúa carrera una sola vez aunque se ejecute múltiples veces	scored_rounds impide doble puntuación	PASS
P-P02	AutoScore puntúa sprint separado de la carrera	Tipo 'sprint' en scored_rounds independiente	PASS
P-P03	Puntos se acumulan correctamente en fantasy_driver_points	ON DUPLICATE KEY UPDATE suma correctamente	PASS

P-P04	Al editar equipo se borran los puntos de pilotos transferidos	clearSelections limpia fantasy_driver_points	PASS
P-P05	Puntuación en batch de 5 equipos procesa todos	Do-while LIMIT/OFFSET completa todos	PASS

Módulo de Seguridad

ID	Prueba	Resultado obtenido	PASS/FAIL
P-S01	Acceso a `/fantasy` sin login redirige a login	Redirect a auth/login	PASS
P-S02	Acceso a `/admin/recalculate` por usuario normal devuelve 403	HTTP 403 Forbidden	PASS
P-S03	Intento de inyección SQL en campo email	Prepared statement escapa el input	PASS
P-S04	Cambio de rol en BD se refleja en siguiente petición sin re-login	refreshUserSession() actualiza sesión	PASS

8. REGISTRO DE INCIDENCIAS

Las incidencias detectadas durante el desarrollo y sus resoluciones se documentan a continuación siguiendo el formato establecido en el punto 3.1.

ID	Fecha	Módulo	Tipo	Gravedad	Descripción	Responsable	Estado
INC-01	2026-03-15	Fantasy	Bug	Alta	Los puntos de la última carrera se mostraban en equipos creados después de dicha carrera	Oscar Ongil	Cerrada
INC-02	2026-03-22	Carreras	Bug	Media	La página de detalle de una carrera sprint	Oscar Ongil	Cerrada

					mostraba los resultados de la carrera principal		
INC-03	2026-04-05	Puntuación	Crítico	Crítica	Race condition: dos procesos simultáneos podían puntuar la misma ronda dos veces	David González	Cerrada
INC-04	2026-04-10	Ligas	Bug	Media	Al expulsar a un miembro de una liga, su equipo fantasy quedaba huérfano en la BD	David González	Cerrada
INC-05	2026-04-10	General	Mejora	Baja	No se informaba al usuario cuando la API de F1 no estaba disponible	Oscar Ongil	Cerrada
INC-06	2026-04-18	Admin	Bug	Media	Los botones de administración no aparecían después de añadir la columna `role` a la BD	Oscar Ongil	Cerrada
INC-07	2026-05-10	Admin	Bug	Media	El admin podía recalcular precios múltiples veces para la misma carrera	David González	Cerrada

Detalle de resoluciones

INC-01 — Puntos de carrera anterior en equipo nuevo

- ***Causa raíz:*** El controlador no comparaba la fecha de la carrera con la fecha de creación del equipo.
- ***Solución:*** En `FantasyController::index()` se compara `$lastRace['date']` con `substr($team['created_at'], 0, 10)`. Si la carrera es anterior al equipo, se suprime `$lastRaceName`.

INC-02 — Sprint muestra resultados de carrera

- ***Causa raíz:*** `RacesController::show()` siempre cargaba resultados de carrera independientemente del parámetro `?type=sprint`.
- ***Solución:*** Se añadió detección de `$_GET['type']`. Si es 'sprint', se llama a `$this->f1->getSprintResults()` en lugar de `getRaceResults()`.

INC-03 — Doble puntuación por race condition

- ***Causa raíz:*** El flujo marcaba la ronda como puntuada después de ejecutar la puntuación, dejando una ventana temporal en la que otro proceso podía pasar la validación.
- ***Solución:*** Se invierte el orden: se inserta en `scored_rounds` ANTES de puntuar usando `INSERT IGNORE` + comprobación de `affected_rows > 0`. Solo el proceso que insertó la fila procede con la puntuación.

INC-04 — Equipo huérfano al expulsar miembro

- ***Causa raíz:*** `LeagueController::kick()` solo eliminaba la entrada de `league_members` sin eliminar el equipo asociado.
- ***Solución:*** Se añadió llamada a `$this->ftm->deleteTeamByUser($userId, $leagueId)` que elimina en cascada el equipo, sus selecciones y sus puntos acumulados. También se añadió `cleanupOrphanedTeams()` en `AutoScore`.

INC-05 — Sin aviso de API no disponible

- ***Causa raíz:*** Los fallos de `file_get_contents` se ignoraban silenciosamente.
- ***Solución:*** Se añadió propiedad `$apiError` en `F1ApiModel`. Se activa cuando falla una llamada HTTP. Los controladores lo pasan a la vista, y el layout muestra un banner de aviso cuando está activo.

INC-06 — Botones admin no visibles tras añadir columna `role`

- ***Causa raíz:*** Las sesiones existentes no tenían la clave `role` porque se crearon antes de añadir la columna.
- ***Solución:*** Se implementó `Controller::refreshUserSession()` que re-lee el campo `role` de la BD en cada petición protegida (`requireLogin()`), manteniendo el rol siempre actualizado sin necesidad de re-login.

INC-07 — Recalcular precios múltiples veces

- ***Causa raíz:*** No existía protección contra la ejecución repetida del recalcular de precios.
- ***Solución:*** Se usa la tabla `scored_rounds` con tipo 'recalculate'. Antes de ejecutar y tras ejecutar se verifica/registra en esta tabla. El ENUM de la columna `type` se amplió para incluir el nuevo valor.

9. REGISTRO DE CAMBIOS

ID Cambio	Fecha	Tipo	Descripción	Solicitante	Impacto	Decisión
CHG-01	2026-03-10	Mejora	Añadir campo `league_id` a `fantasy_teams` para soporte multi-liga	Equipo	Alto: migración de BD y refactor de modelos	Aprobado e implementado

CHG-02	2026-03-28	Nueva funcionalidad	Añadir tabla `fantasy_driver_points` para acumular puntos por selección	Equipo	Medio: nueva tabla + método `getSelectionPoints()`	Aprobado e implementado
CHG-03	2026-04-02	Mejora	Añadir columna `role` a tabla `users` y reemplazar `ADMIN_EMAIL`	Equipo	Bajo: migración SQL + actualizar sesiones	Aprobado e implementado
CHG-04	2026-04-10	Refactoring	Eliminar vista y rutas de `admin/score` — AutoScore gestiona la puntuación	Equipo	Bajo: simplificación de interfaz	Aprobado e implementado
CHG-05	2026-04-15	Mejora	Implementar procesamiento en batch de 5 equipos en `scoreRound()`	Equipo	Bajo: mejora de rendimiento y memoria	Aprobado e implementado
CHG-06	2026-04-20	Mejora	Añadir soporte sprint al calendario (eventos separados) y página de detalle	Equipo	Medio: vistas y controlador	Aprobado e implementado
CHG-07	2026-05-01	Mejora	Limitar a LIMIT 5 las consultas de ranking en vistas	Equipo	Bajo: ajuste en modelos	Aprobado e implementado
CHG-08	2026-05-10	Mejora	Añadir botón de edición de equipo y vista `fantasy/edit`	Oscar Ongil	Bajo: nueva vista y ruta	Aprobado e implementado

10. DESPLIEGUE DEL PROYECTO

10.1. Despliegue en servidor, incluyendo base de datos

Requisitos previos

- Docker Desktop instalado y en ejecución (versión 24.x o superior)
- Docker Compose incluido (v2)
- Puertos 8080, 8081 y 3307 libres en el host

Estructura del entorno Docker

El archivo `docker-compose.yml` define tres servicios conectados en la red `f1net`:

Servicio	Imagen	Puerto host	Puerto interno
`apachephp` (f1fantasy_web)	Dockerfile personalizado PHP 8.2 + Apache 2.4	8080	80
`mariadb` (f1fantasy_db)	mariadb:10.11	3307	3306

`phpmyadmin` (f1fantasy_pma)	phpmyadmin:5	8081	80
---------------------------------	--------------	------	----

Variables de entorno del contenedor web

Variable	Valor	Descripción
`DB_HOST`	`mariadb`	Nombre del servicio MariaDB en la red interna
`DB_PORT`	`3306`	Puerto interno de la BD
`DB_USER`	`f1user`	Usuario de la base de datos
`DB_PASS`	`f1pass`	Contraseña del usuario
`DB_NAME`	`f1fantasy`	Nombre de la base de datos

Volúmenes persistentes

Volumen	Montaje	Contenido
`f1_dbdata`	`/var/lib/mysql`	Datos de MariaDB (persistente entre reinicios)
`f1_cache`	`/var/www/html/cache`	Caché JSON de la API (1 hora TTL)

El código fuente ([app/](#), [config/](#), [core/](#), [public/](#)) se monta como volúmenes en tiempo real, por lo que **no es necesario reconstruir la imagen para reflejar cambios en el código.**

Proceso de despliegue

Paso 1 — Clonar o copiar el proyecto

```
Copiar la carpeta docker-flfantasy al servidor destino.
```

Paso 2 — Levantar los contenedores

```
docker-compose up -d --build
```

Este comando:

- Construye la imagen [apachehttpd](#) desde [docker/apachehttpd/Dockerfile](#).
- Inicia MariaDB. El script [config/schema.sql](#) se ejecuta automáticamente en el primer arranque gracias al directorio [docker-entrypoint-initdb.d](#).
- El contenedor web espera a que MariaDB esté sano (healthcheck) antes de arrancar.

Paso 3 — Aplicar migraciones adicionales

Si la base de datos ya existía (primer arranque sin [schema.sql](#)), aplicar las migraciones manualmente:

```
docker exec -i flfantasy_db mysql -u fluser -pflpass flfantasy <
config/migration_market.sql
```

Paso 4 — Verificar el despliegue

Comprobar que los tres contenedores están en estado [healthy](#) o [running](#):

```
docker-compose ps
```

Paso 5 — Configurar usuario admin

Ejecutar en la base de datos para asignar el rol de administrador al usuario deseado:

```
UPDATE users SET role = 'admin' WHERE email = 'correo_admin@ejemplo.com';
```

Accesible desde phpMyAdmin en <http://localhost:8081> (usuario: `root`, contraseña: `root`).

Detener el entorno

```
docker-compose down
```

Para eliminar también los volúmenes (datos incluidos):

```
docker-compose down -v
```

10.2. Conexión en cliente

Acceso desde navegador local

Una vez levantados los contenedores, la aplicación está disponible en:

Servicio	URL
Aplicación F1 Fantasy	<code>`http://localhost:8080`</code>
phpMyAdmin (administración BD)	<code>`http://localhost:8081`</code>
MariaDB directo (cliente SQL)	<code>`localhost:3307`</code> (usuario: <code>`f1user`</code> , contraseña: <code>`f1pass`</code>)

Acceso desde otra máquina en la red

Sustituir `localhost` por la IP del servidor donde corre Docker. Por ejemplo:

```
http://192.168.1.X:8080
```

No se requiere configuración adicional en el cliente — solo un navegador web moderno.

Notas de compatibilidad de navegadores

La aplicación ha sido probada en:

- Google Chrome 124+
- Mozilla Firefox 125+
- Microsoft Edge 124+

No requiere plugins ni extensiones. JavaScript habilitado para los formularios de equipo (sin él, la funcionalidad básica de navegación y lectura es accesible).

11. MANUAL DE USO

11.1. Para usuarios

Registro y acceso

- Acceder a <http://localhost:8080>.
- Hacer clic en **Registrarse** (esquina superior derecha).
- Introducir nombre de usuario, email y contraseña (mínimo 6 caracteres).
- Confirmar la contraseña y enviar el formulario.

- Iniciar sesión con las credenciales registradas.

Explorar datos de F1

- **Pilotos** (/drivers): ver todos los pilotos de la temporada actual con su precio de mercado actual, puntos en el campeonato y color de escudería. Hacer clic en un piloto para ver su detalle.
- **Equipos** (/teams): ver constructores con precio de mercado.
- **Calendario** (/races): ver todos los Grandes Premios de la temporada. Los fines de semana con sprint aparecen como dos eventos separados: la carrera principal y el sprint. Hacer clic en cualquiera para ver sus resultados.

Crear y gestionar una liga

- Ir a **Ligas** en el menú principal.
- Hacer clic en **Crear liga**.
- Asignar un nombre, descripción opcional y elegir si es pública o privada.
- La liga se crea con un **código de invitación** único. Compartir este código con otros jugadores para que se unan.
- Para unirse a una liga existente:
 - **Liga pública**: hacer clic en **Unirse** directamente desde el listado.
 - **Liga privada**: introducir el código en el formulario "Unirse con código" de la página de ligas.
- Para **abandonar una liga**: acceder al detalle de la liga y hacer clic en **Salir**.
- El **creador de la liga** puede expulsar miembros desde la página de detalle de la liga.

Crear el equipo fantasy

- Desde el detalle de una liga en la que seas miembro, hacer clic en **Crear equipo**.
- Introducir el nombre del equipo.
- Seleccionar exactamente **5 pilotos y 2 constructores**.
- El contador de presupuesto se actualiza en tiempo real — no puede superar **100M€**.
- Confirmar con **Guardar equipo**.

Ver el equipo

- Desde el detalle de la liga o mediante el menú de perfil, acceder al equipo.
- La pantalla muestra:
 - **Nombre del equipo y presupuesto invertido**.
 - **Puntos totales acumulados** desde la creación del equipo.
 - Para cada piloto/constructor: precio de mercado actual · puntos acumulados desde la creación.
 - **Puntos de la última jornada** (carrera y sprint si los hubo), siempre que la carrera ocurriera después de la creación del equipo.

Editar el equipo

- En la página de equipo, hacer clic en **Editar equipo**.
- Modificar los pilotos y/o constructores (se muestran los actuales seleccionados).
- El presupuesto no puede superarse.
- Los puntos acumulados de los pilotos transferidos se resetean.

Ranking

- **Ranking de liga:** visible en el detalle de cada liga, mostrando el top 5 de equipos dentro de esa liga.
- **Ranking global** (</fantasy/ranking>): top 5 de todos los equipos de la plataforma.

11.2. Para administradores

Un administrador es un usuario con `role = 'admin'` en la base de datos. Los administradores disponen de un panel flotante accesible desde cualquier página (icono  en la esquina inferior derecha).

Asignar rol de administrador

Desde phpMyAdmin (<http://localhost:8081>):

```
UPDATE users SET role = 'admin' WHERE email = 'email_del_admin@ejemplo.com';
```

El cambio se refleja en la siguiente petición del usuario afectado, sin necesidad de que cierre sesión.

Recalcular precios de mercado

- Hacer clic en el icono  (panel admin flotante).
- Seleccionar **Recalcular precios**.
- La pantalla muestra la última carrera completada y la fórmula de cálculo.
- Si los precios ya fueron recalculados para esa carrera, se mostrará un aviso y el botón estará bloqueado. **Solo se puede recalcular una vez por carrera completada.**
- Al confirmar, se aplica la fórmula a todos los pilotos y constructores con precio en la tabla `market_prices` y se muestra un resumen de los cambios.

Fórmula de recalcular:

```
ΔP_perf = 0.05 × (puntos_reales - precio_actual × 0.7)
ΔP_mkt = 1.5 × (compras - ventas) / total_usuarios
ΔP_total = clamp(ΔP_perf + ΔP_mkt, -1.5, +1.5)
Constructores: ΔP_total × 0.8
Límites: 4.0M€ - 35.0M€
```

Gestión directa de la base de datos

phpMyAdmin está disponible en <http://localhost:8081> con las credenciales de root. Permite:

- Consultar y modificar cualquier tabla.
- Exportar/importar datos.
- Ejecutar SQL personalizado.
- Supervisar el estado de las tablas.

12. CONCLUSIONES

12.1. Síntesis de resultados

El proyecto **Fantasy F1 Manager** ha sido completado satisfactoriamente, alcanzando la totalidad de los casos de uso planificados y superando las pruebas funcionales sin fallos en la versión final entregada.

Los principales logros técnicos del proyecto son:

Robustez del sistema de puntuación. La implementación del mecanismo atómico con `INSERT IGNORE + affected_rows` sobre la tabla `scored_rounds` garantiza que cada ronda se puntúa exactamente una vez, independientemente de cuántas peticiones web concurrentes activen el AutoScore. Este patrón es equivalente a un mutex distribuido a nivel de base de datos.

Sistema de precios de mercado dinámico. Los precios fluctúan en función del rendimiento real del piloto/constructor en la última carrera y de la demanda de mercado (compras vs ventas de todos los usuarios). La fórmula incluye un factor amortiguador para constructores y límites absolutos para garantizar la estabilidad del juego.

Arquitectura multi-liga. Los equipos existen dentro del contexto de una liga, lo que permite a un mismo usuario participar en múltiples ligas con equipos independientes. Los puntos acumulados se rastrean por equipo y por selección, permitiendo comparaciones justas independientemente de cuándo se creó el equipo.

Integración de APIs externas con resiliencia. El sistema utiliza dos APIs externas (Jolpica y OpenF1) con un sistema de caché de archivos. En caso de fallo de la API, la aplicación degrada graciosamente: muestra los datos de la caché y un banner informativo, sin interrumpir la experiencia de usuario.

Despliegue reproducible con Docker. Todo el stack (web, base de datos, administración) se levanta con un único comando (`docker-compose up -d --build`) en cualquier máquina con Docker instalado.

12.2. Líneas futuras de trabajo

Las siguientes mejoras se identifican como extensiones naturales del proyecto para futuras iteraciones:

- 1. Mercado de fichajes con ventanas temporales.** Limitar los traspasos a periodos concretos (por ejemplo, solo entre carreras) para añadir profundidad estratégica.
 - 2. Notificaciones en tiempo real.** Implementar WebSockets o Server-Sent Events para notificar a los usuarios cuando se puntúa una carrera o cuando un piloto cambia de precio significativamente.
 - 3. Estadísticas históricas de piloto/constructor.** Mostrar gráficas de evolución de precio y puntos a lo largo de la temporada, utilizando la tabla `price_history` ya implementada.
 - 4. Modo torneo con eliminatorias.** Añadir fases eliminatorias entre ligas para determinar un campeón absoluto de la plataforma.
 - 5. Despliegue en nube.** Migrar de Docker en local a un entorno cloud (AWS, Azure o similar) con HTTPS, dominio personalizado y base de datos gestionada para permitir acceso público.
 - 6. Aplicación móvil.** Desarrollar una PWA (Progressive Web App) o una aplicación nativa que consuma una API REST expuesta por el backend, permitiendo gestionar el equipo desde el móvil.
 - 7. Sistema de logros y badges.** Implementar un sistema de recompensas por hitos (primera liga ganada, mejor fichaje de la temporada, etc.) para aumentar la retención y motivación de los usuarios.
 - 8. Pruebas automatizadas.** Implementar una suite de pruebas unitarias y de integración con PHPUnit, cubriendo al menos el motor de puntuación, el cálculo de precios y el sistema de autenticación, para garantizar la no regresión en futuras entregas.
-